

**CS5600 Fall 2025 Section 3**

**Final Exam**

**Duration: 100 minutes**

**Examiner: Xiang Ren**

Name:

NUID:

**No exam aids are allowed.** This exam is closed book and notes. You may not use any electronics.

The exam consists of 14 questions and 10 pages. The maximum grade possible is 100.

Please write your name and NUID on each page. Do not remove any pages from the booklet.

Write your answers clearly and concisely. You will not be rewarded for long answers that are not to the point. Make sure your writing is legible. Marks will be deducted for wrong answers. Write down any assumptions.

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| Q1 | Q2 | Q3 | Q4 | Q5 | Q6 | Q7 |
| /7 | /7 | /2 | /4 | /3 | /3 | /2 |

|    |    |     |     |     |     |     |
|----|----|-----|-----|-----|-----|-----|
| Q8 | Q9 | Q10 | Q11 | Q12 | Q13 | Q14 |
| /3 | /2 | /15 | /20 | /12 | /10 | /10 |

Total: /100

1. [lab3, 7 marks]

a) [2 marks] What is the name of the function used to create a thread in lab3? (No need to write the full signature)

b) [2 marks] What does it mean when `kv->keys[...].stat` is 1 versus when it is 0?

c) [1 marks] Does `kv_read` and `kv_write` need to be atomic?

d) [2 marks] What should your program do when it tries to fetch from the head of the queue but the queue is empty?

2. [lab4, 7 marks]

a) [4 marks] In lab 4, when a process is freed, which of the following are freed?

|                            | Y (Yes) or N (No) |
|----------------------------|-------------------|
| Data pages                 |                   |
| Metadata pages             |                   |
| Page table pages           |                   |
| Secondary page table pages |                   |

b) [1 marks] What is the name of the iterator you use to walk virtual address mappings?

c) [2 marks] What does `PTE_P` and `PTE_U` represent?

[Partitions, 2 marks]

3. What kinds of fragmentation problems plague fixed-sized partitioning and variable-sized partitioning?

Name:

NUID:

[Demand paging, 4 marks]

4. Select all the options that apply:

- a) Every memory access needs to involve the TLB
- b) Every TLB miss is accompanied with a page fault
- c) Every page fault is accompanied with a TLB miss
- d) Demand paging only occurs when physical frames are in high demand
- e) When a process is created, the page table entries created have the valid/present bit marked 1
- f) If the dirty bit is marked 1 for a page table entry, any access to the page will cause a page fault

[Disk scheduling, 3 marks]

5. Which scheduling algorithm is optimized for performance and which is optimized for fairness?

- a) Shortest Positioning Time First for performance and C-SCAN for fairness
- b) FIFO for performance and SCAN for fairness
- c) SCAN for performance and Shortest Seek Time First for fairness
- d) None of the above

[RAID, 2 marks]

6. How many disk failures can you recover from in a RAID4 array? How about at RAID1?

[Journalling, 3 marks]

7. Which of the following statements are true?

- a) Journalling and checkpointing metadata for the same transaction can be done concurrently without causing corruption
- b) Journalling ensures that there is no data loss
- c) Committing a transaction involves writing exactly 1 block
- d) Journalling all the metadata blocks of a transaction can be done concurrently

Name:

NUID:

[Log-structured file system, 3 marks]

8. Which of the following statements are true?

- a) The log-structured file system is optimized for sequential writes
- b) The log-structured file system is optimized for random writes
- c) The log-structured file system writes in random order
- d) The log-structured file system writes in sequential order
- e) The log-structured file system reduces disk seek
- f) The log-structured file system worsens wear leveling

[Security, 2 marks]

9. Which access control models are used by a Unix operating system?

- a) Access control bits
- b) Capabilities
- c) Both

[Synchronization, 15 marks]

10. Your friend created their own custom library for condition variables, but they noticed that the implementation doesn't work, can you help them debug the implementation of `cond_wait`?

```
void cond_signal(cond *);
```

```
void cond_wait(cond *, pthread_mutex_t *);
```

```
cond_wait(cond *cv, pthread_mutex_t *mutex) {
    enqueue(cv->q, gettid());
    sleep();
    pthread_mutex_unlock(mutex);
}
```

a) [3 marks] Assume the user application has properly acquired mutex before calling `cond_wait`, what would happen when the user application calls `cond_wait`?

After you explained to your friend the problem with their implementation, they changed the code to:

```
cond_wait(cond *cv, pthread_mutex_t *mutex) {
    enqueue(cv->q, gettid());
    pthread_mutex_unlock(mutex);
    sleep();
}
```

b) [3 marks] Is this implementation now correct? If yes, explain why. If no, explain how you would fix it (no need to write actual code).

Name:

NUID:

c) [9 marks] Now that your condition variable implementation works, create a custom semaphore implementation using your condition variable implementation.

//Useful function signatures

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

//Usage:

```
pthread_mutex_t lock; //declares a lock
```

```
pthread_mutex_init(&lock); //initializes the lock
```

```
cond cv; //declares a condition variable
```

```
cond_init(&cv, NULL); // initializes the condition variable
```

```
// Here you may declare and initialize variables, including those that hold states for sem_wait()
and sem_post().
```

```
sem_wait() { //P()
```

```
}
```

```
sem_post { //V()
```

```
}
```

Name:

NUID:

[Memory management, 20 marks]

11. You are designing a strange memory management system for a 64bit operating system, where the page table entry is only 2 bytes or 16 bits. Each virtual page/physical frame is still 4KB and each page table must fit in a 4KB page. Given that for each page table entry, you need to reserve one bit for the present/valid bit, one bit for the referenced bit, and one bit for the dirty bit, and 3 bits for read, write, and execute permissions.

a) [5 marks] What is the maximum amount of physical memory that your system can utilize? Show calculations and/or explanations.

b) [5 marks] You decided that you only want to have 2 levels of page tables. What is the maximum address space size? Show calculations and/or explanations.

c) [5 marks] Within a single process's virtual address space, what is the maximum possible number of page tables needed to contain the mappings to all physical frames? Show calculations and/or explanations.

d) [5 marks] Within a single process's virtual address space, what is the minimal possible number of page tables needed to contain the mappings to all physical frames? Show calculations and/or explanations.

[Page replacement, 12 marks]

12. Assume the following virtual pages (a, b, ..., h) are accessed at the given timestamps (t=):

a[t=100] b[t=101] c[t=102] d[t=103] e[t=104] f[t=105] g[t=106] h[t=107]

and the following page faults happened at the given timestamps, where each page fault caused a page eviction, but none of the pages a–h is evicted.

page\_fault0[t=100.5] page\_fault1[t=101.5] page\_fault2[t=103.5] page\_fault3[t=107.5]

At page\_fault3, pages a to h are all resident in memory, and the OS needs to evict one of them.

Questions a, b, and c ask you to identify the full set of pages that could be chosen as the victim for each given algorithm. Write down any assumptions.

a) [2.5 marks] Using the LRU replacement algorithm.

b) [2.5 marks] Using the Clock replacement algorithm.

c) [2.5 marks] Using Working Set Clock (WSClock) replacement algorithm, with a working set interval of 5 (same unit as the timestamps above).

d) [2.5 marks] If you want WSClock to make the same eviction decisions as LRU, how would you change the working set interval?

e) [2 marks] Which algorithm is the most lightweight to implement? Why?

Name:

NUID:



[File systems, 10 marks]

13. You are modifying a Unix file system and decided to limit the amount of metadata each file needs to hold and remove the triple indirect pointer, allowing only 12 direct pointers, 1 single indirect pointers, and 1 double indirect pointer. However, you still want to be able to support the maximum file size that a file with a triple indirect pointer can support, so you need to allocate data blocks contiguously. Assume each block is 4KB and each block pointer is 4B.

a) [4 marks] How many metadata blocks could you save at most? Show calculations. No need to compute the final numerical answer.

b) [4 marks] How many blocks of files would you need to allocate contiguously each time? Show calculations. No need to compute the final numerical answer.

c) [2 marks] What is the problem with this file system design?

[File system, 10 marks]

14. You are creating a small 1KB file "c" in "/a/b/" (and "/a/b/" already exists) in a Unix file system with the following metadata:

a) [6 marks] Which of these pieces of metadata would you need to update to create this file, and what do you update in each?

inode table, inode bitmap, data bitmap, directory entries

b) [4 marks] Now in a separate thread, you want to concurrently create file "e" in "/a/d/". However, you noticed that the two file creations are completely serialized, and inferred that the file system must use a single global lock to protect all updates, i.e., each file system call that can update the file system acquires the same lock in the beginning and releases it at the end of the system call. You wanted to suggest an optimized implementation that allows more parallelism, how would you design the locking scheme differently?